

College Examination System

Submitted by

Abhinandan Rakshit (13031123001)

Abhraneel Dhar (13031123002)

Under the guidance of
Prof. Poulami Dutta
Asst. Professor, Department of CSE

Submitted for the partial fulfillment for the course on
Database Management Systems (PCC-CSBS502/592)

Techno India,
EM 4/1, Salt Lake, Sector V, Kolkata – 700091



DEPARTMENT OF COMPUTER SCIENCE AND BUSINESS SYSTEMS

CERTIFICATE

This is to certify that the mini project report entitled “College Examination System” submitted by Abhinandan Rakshit and Abhraneel Dhar of B.Tech (CSBS), Techno Main Salt Lake, has been carried out under my supervision and guidance. The work embodied in this report is original and has not been submitted elsewhere.

Guide:

(Signature)

Name: _____ Prof. Poulami Dutta _____

Designation: _____ Asst. Professor _____

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Prof. Poulami Dutta, faculty for the course on Database Management Systems (PCC-CSBS502/592), from the Department of Computer Science and Engineering, whose role was invaluable for this mini project. We are extremely thankful for the keen interest she took in advising us, for the books and reference materials provided and for the moral support extended to us. We would also like to thank the faculty members of the Department of Computer Science and Business Systems for their guidance and support throughout the completion of this mini project.

Place: Techno Main, Salt Lake

Date:

.....

.....

ABSTRACT

This project report presents the design and implementation of a College Examination System, a real-world application developed using Database Management System (DBMS) concepts.

The system facilitates the management of students, faculty, departments, exams, questions, and results in a structured and efficient manner.

The report details the database design, including entity-relationship modeling, relational schema creation, and enforcement of integrity constraints such as primary keys, foreign keys, and domain-specific validations. Normalization techniques were applied to ensure data consistency and minimize redundancy. SQL queries were implemented for CRUD operations, exam management, result computation, and reporting. Additionally, the project includes an application interface that allows administrators, HODs, and students to interact with the system, conduct exams, record answers, and generate results.

The system provides a robust framework for automating examination processes and enhancing accuracy, efficiency, and transparency in academic assessments.

Contents

CERTIFICATE	2
ABSTRACT	4
1. Introduction	7
1.1 Overview of the Problem Domain	7
1.2 Motivation for the Project	7
1.3 Scope and Objectives	7
1.4 Relevance to Real-Life	8
2. Literature Review	9
2.1 Summary of Existing Systems or Related Works	9
2.2 Limitations of Current Approaches	9
2.3 Research Gap Identification	9
3. Problem Definition and Objectives	10
3.1 Problem Statement	10
3.2 Objectives of the Project	10
3.3 Expected Outcomes	10
4. System Analysis	11
4.1 Requirements Analysis	11
4.1.1 Functional Requirements	11
4.1.2 Non-Functional Requirements	11
4.2 Feasibility Study	12
4.2.1 Technical Feasibility	12
4.2.2 Economic Feasibility	12
4.2.3 Operational Feasibility	13
5. System Design	14
5.1 Database Design	14
5.1.1 Entity–Relationship (ER) Diagram	14
5.1.2 Relational Schema	15
5.2 Implementation Tools and Technologies Used	16
Frontend	16
Backend	16
Database	16
Development Tools	16
5.3 Frontend and Backend Description	17

5.3.1 Frontend Description	17
5.3.2 Backend Description.....	17
5.4 Code Snippets	20
Login Code.....	20
JWT Login Code.....	20
Department Actions Code.....	22
Exam Actions Code	23
Student Actions Code	41
User Actions Code	45
6. Testing and Evaluation	47
6.1 Screenshots of the Application	47
6.2 Output Demonstration.....	53
6.2.1 Admin Module Output.....	53
6.2.2 Student Module Output.....	53
6.2.3 Backend Output	53
6.3 Analysis of Results	53
6.3.1 Functional Performance	53
6.3.2 Usability Evaluation.....	53
6.3.3 Security Evaluation.....	54
6.3.4 Performance Stability.....	54
6.3.5 Overall Evaluation	54
7. Conclusion	55
7.1 Summary of Achievements.....	55
7.2 Limitations	55
8. Future Scope	56
8.1 Future Enhancements.....	56
9. References	57
10. Appendix.....	58
10.1 Acronyms.....	58
10.2 Code Snippets	58
10.2.1 MySQL Connection Setup (Next.js).....	58
10.2.2 JWT Token Generation.....	58
10.2.3 Admin Login API Example	58
10.2.4 Exam Result Calculation Snippet	59

1. Introduction

1.1 Overview of the Problem Domain

Educational institutions conduct numerous examinations every semester, involving tasks such as creating question papers, enrolling students, managing departments, evaluating responses, and preparing results. When handled manually, these processes are time-consuming, error-prone, and difficult to track. Traditional paper-based systems also lack transparency and require extensive administrative effort.

A digital examination management system helps streamline these activities by centralizing information, automating workflows, and improving accessibility for students and administrators.

1.2 Motivation for the Project

Most colleges face challenges such as maintaining accurate exam records, avoiding scheduling conflicts, ensuring secure student data, and generating timely results. Manual processes often lead to inefficiencies and mismanagement.

This project was motivated by the need to build a secure, modern, and scalable solution that simplifies these operations. By integrating full-stack technologies and database concepts, the system demonstrates how technology can reduce administrative load and enhance overall examination management.

1.3 Scope and Objectives

Scope:

The system focuses on managing departments, students, exams, and results in a structured manner. It provides separate access levels for administrators and students to ensure secure and smooth operation.

Objectives:

- Design a well-structured relational database using ER modeling and normalization.
- Implement a full-stack web application using Next.js, TypeScript, MySQL, and JWT authentication.
- Enable administrators to create, edit, delete, and manage exams, departments, and student records.
- Allow students to register, take exams online, and view their results securely.
- Generate and store exam results automatically through backend logic.
- Provide an easy-to-use and responsive interface for both roles.

1.4 Relevance to Real-Life

Examination management is a critical component in any educational institution. A digital system reduces paperwork, improves accuracy, minimizes human errors, and enhances transparency. Such systems are widely used in colleges, competitive exams, online learning platforms, and certification bodies. The technologies and database principles applied here closely mirror real-world software solutions, making the project practical, industry-relevant, and beneficial for modern academic environments.

2. Literature Review

2.1 Summary of Existing Systems or Related Works

Several online examination and academic management systems have been developed to automate institutional workflows. Widely used platforms such as **Moodle**, **Google Classroom**, and various proprietary college ERP suites provide features like online tests, question banks, grade recording, and student management.

Research papers and academic projects in this domain commonly explore database-driven systems that allow instructors to create exams, evaluate student performance, and maintain records through a centralized interface. Many systems also implement role-based access to ensure secure operations between students, teachers, and administrators.

2.2 Limitations of Current Approaches

Despite being functional, existing solutions face several limitations:

- **Complexity:** Many ERP or LMS platforms are feature-heavy and difficult for smaller institutions to adopt.
- **Lack of customization:** Ready-made systems are often rigid, making it challenging to modify workflows according to a college's specific needs.
- **High cost:** Commercial solutions require licensing fees or subscription charges that smaller colleges may find difficult to afford.
- **Limited exam flexibility:** Some systems do not support customizable exam patterns, dynamic question creation, or instant result generation.
- **Security concerns:** Password-based logins without modern authentication (e.g., JWT) can lead to unauthorized access incidents.
- **Performance issues:** Some platforms struggle during peak loads when many students attempt exams simultaneously.

2.3 Research Gap Identification

Based on the limitations of existing platforms, several gaps remain unaddressed:

- A need for a **lightweight, customizable examination system** specifically suited for college-level usage.
- A system built with **modern full-stack technologies** (Next.js + TypeScript + MySQL) rather than traditional outdated stacks.
- Simple, intuitive interfaces that reduce the learning curve for both admins and students.
- Stronger **security measures** such as JWT-based authentication and protected API routes.
- Real-time result generation and accurate performance tracking without requiring external modules.

3. Problem Definition and Objectives

3.1 Problem Statement

Many colleges still rely on traditional methods for conducting examinations, managing student records, and evaluating performance. These manual workflows often lead to inefficiencies such as time-consuming exam setup, difficulty in generating results, errors in maintaining records, and limited transparency for students.

Existing digital solutions are either too complex, costly, or not customizable enough to fit the requirements of small and medium-sized educational institutions.

There is a need for a secure, user-friendly, and fully digital examination system that streamlines administration tasks and provides students with a smooth exam-taking experience.

3.2 Objectives of the Project

1. **To design and develop an online examination system** using modern full-stack technologies (Next.js, TypeScript, MySQL).
2. **To create a secure authentication system** using JWT for both admin and student roles.
3. **To enable admins to efficiently manage departments, students, and examinations**, including adding, editing, and deleting records.
4. **To provide students with an easy-to-use exam interface** where they can attempt tests and view results.
5. **To ensure accurate and instant evaluation** of student responses and automatic result generation.
6. **To implement a structured relational database** that maintains data integrity, supports efficient queries, and ensures reliable long-term storage.
7. **To build a scalable system** that can be extended in the future with additional features such as analytics, notifications, or auto-proctored exams.

3.3 Expected Outcomes

- A fully functional **web-based examination platform** accessible to students and administrators.
- Secure, role-based access control using JWT authentication.
- Smooth workflows for exam creation, question management, student registration, and department management.
- A responsive **student exam interface** with real-time timer and question navigation.
- Automated result computation with accuracy in scoring and reporting.
- A robust **MySQL database** with properly defined relationships, constraints, and integrity rules.
- A clean, scalable architecture suitable for deployment and future enhancements.

4. System Analysis

4.1 Requirements Analysis

4.1.1 Functional Requirements

The system must support all core operations required by administrators and students.

Admin Functional Requirements

1. **Admin Login** using email and password with JWT-based authentication.
2. **Manage Departments**: Add, edit, delete department details.
3. **Manage Students**: Register students, update details, and remove records.
4. **Manage Exams**: Create, edit, update, and delete exams.
5. **Manage Questions**: Add and modify questions for each exam.
6. **View Student Attempts** for each exam.
7. **View Results** and generate performance summaries.

Student Functional Requirements

1. **Student Login** via secure JWT authentication.
2. **View Available Exams** assigned to their department.
3. **Attempt Online Examinations** within the provided time slot.
4. **Submit Answers**, which the system stores securely.
5. **View Results** and performance metrics after completion.

System Functional Requirements

1. **Automatic Evaluation** of objective-type questions.
2. **Timer Management** during examination.
3. **Result Calculation** (marks, percentage, correct answers).
4. **Relational Database Storage** ensuring data consistency.
5. **Role-Based Access Control** (Admin, Student).

4.1.2 Non-Functional Requirements

Performance Requirements

- System must handle multiple students taking exams simultaneously.
- Database queries must execute within acceptable response times.

Security Requirements

- All routes protected using **JWT authentication**.
- Passwords must be **hashed** before storage.
- Strict role-based permissions must be enforced.

Usability Requirements

- UI must be clean, responsive, and easy to navigate.
- Students must be able to attempt exams without confusion.

Reliability Requirements

- System must prevent exam submission failures.
- Data integrity must be maintained using constraints and foreign keys.

Scalability Requirements

- Backend should support growing student and exam data.
- Architecture should allow new modules (analytics, auto-proctoring, etc.).

Maintainability Requirements

- Codebase (Next.js + TypeScript) must be modular and organized.
- Database schema must follow normalization rules.

4.2 Feasibility Study

4.2.1 Technical Feasibility

The technology stack Next.js, TypeScript, MySQL, and JWT is modern, widely supported, and suitable for building a secure examination platform.

These technologies are open-source, lightweight, and efficient enough for real-time exam operations.

Both team members possess the necessary skills to develop, deploy, and maintain the system.

Conclusion: *Technically feasible.*

4.2.2 Economic Feasibility

The system uses free and open-source technologies, reducing software cost to zero.

Deployment can be done on low-cost cloud services or institutional servers.

Compared to commercial ERP solutions, the cost is minimal.

Conclusion: *Economically feasible and highly cost-effective.*

4.2.3 Operational Feasibility

Administrators can easily adopt the system for managing departments, students, and exams.

Students require only basic device knowledge to take exams.

The system improves efficiency, reduces workload, and eliminates manual errors.

Conclusion: *Operationally feasible and beneficial to the institution.*

5. System Design

5.1 Database Design

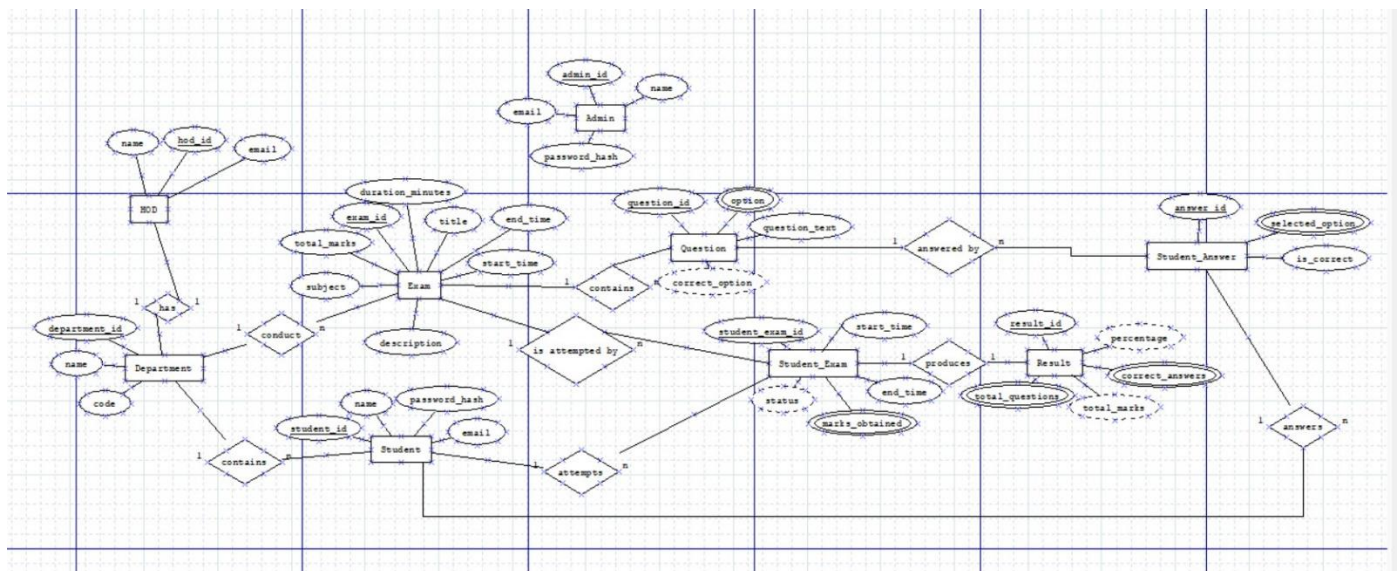
5.1.1 Entity–Relationship (ER) Diagram

The ERD visually represents the entities and relationships of the College Examination System. It includes the following major entities:

- Admin
- Department
- HOD
- Student
- Exam
- Question
- Student_Exam
- Student_Answer
- Result

Each relationship follows cardinality constraints such as 1–M and M–1 depending on the interaction between entities.

Insert ER Diagram (.jpeg) here



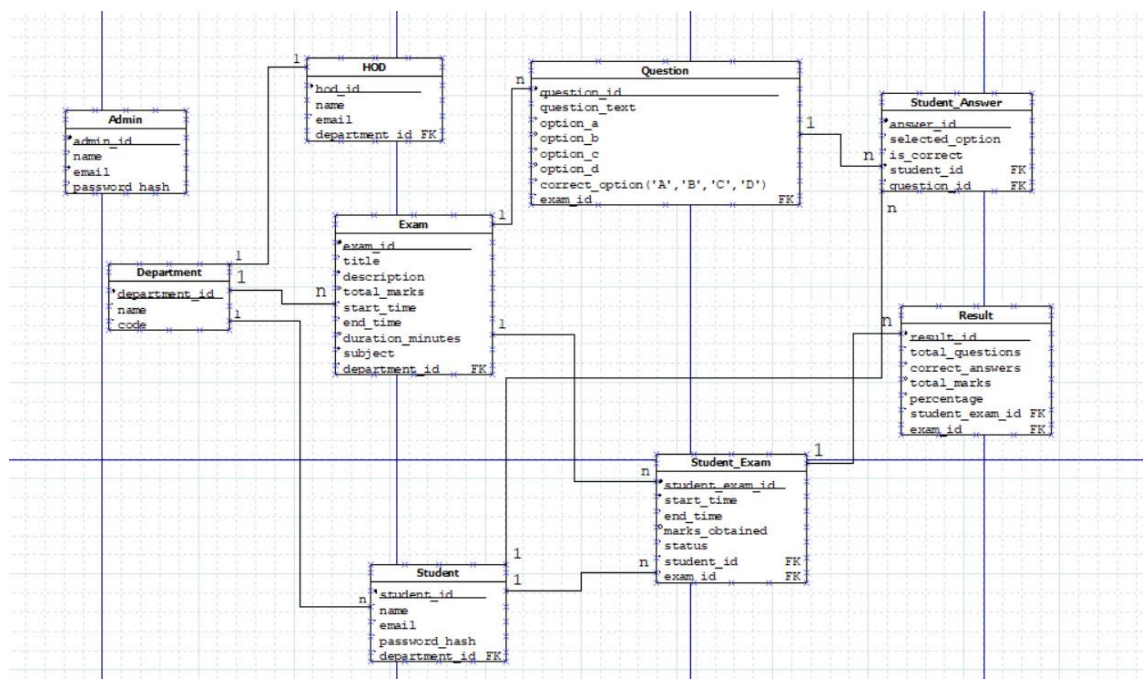
5.1.2 Relational Schema

Below is the relational schema derived from the ERD:

admin(admin_id PK, name, email UNIQUE, password_hash)
department(department_id PK, name UNIQUE, code UNIQUE)
hod(hod_id PK, name, email UNIQUE, department_id FK → department.department_id)
student(student_id PK, name, email UNIQUE, department_id FK → department.department_id, password_hash)
exam(exam_id PK, department_id FK → department.department_id, title, description, total_marks, start_time, end_time, duration_minutes, subject)
question(question_id PK, exam_id FK → exam.exam_id, question_text, option_a, option_b, option_c, option_d, correct_option)
student_exam(student_exam_id PK, student_id FK → student.student_id, exam_id FK → exam.exam_id, start_time, end_time, marks_obtained, status)
student_answer(answer_id PK, student_id FK → student.student_id, question_id FK → question.question_id, selected_option, is_correct)
result(result_id PK, student_exam_id FK → student_exam.student_exam_id, exam_id FK → exam.exam_id, total_questions, correct_answers, total_marks, percentage)

Integrity constraints included:

- **PRIMARY KEYS** ensure unique identification of records.
- **FOREIGN KEYS** enforce referential integrity between dependent tables.
- **UNIQUE constraints** prevent duplicate email, department code, etc.
- **NOT NULL** ensures mandatory fields (like passwords, names).
- **ON DELETE CASCADE** may be applied on foreign keys depending on chosen logic.



5.2 Implementation Tools and Technologies Used

This project uses a full-stack, modern web development stack:

Frontend

- **Next.js 14**
- **TypeScript**
- **React Components**
- **Tailwind**

Backend

- **Next.js Server Actions**
- **TypeScript**
- **JWT Authentication** (JSON Web Tokens)
- **bcrypt** for password hashing

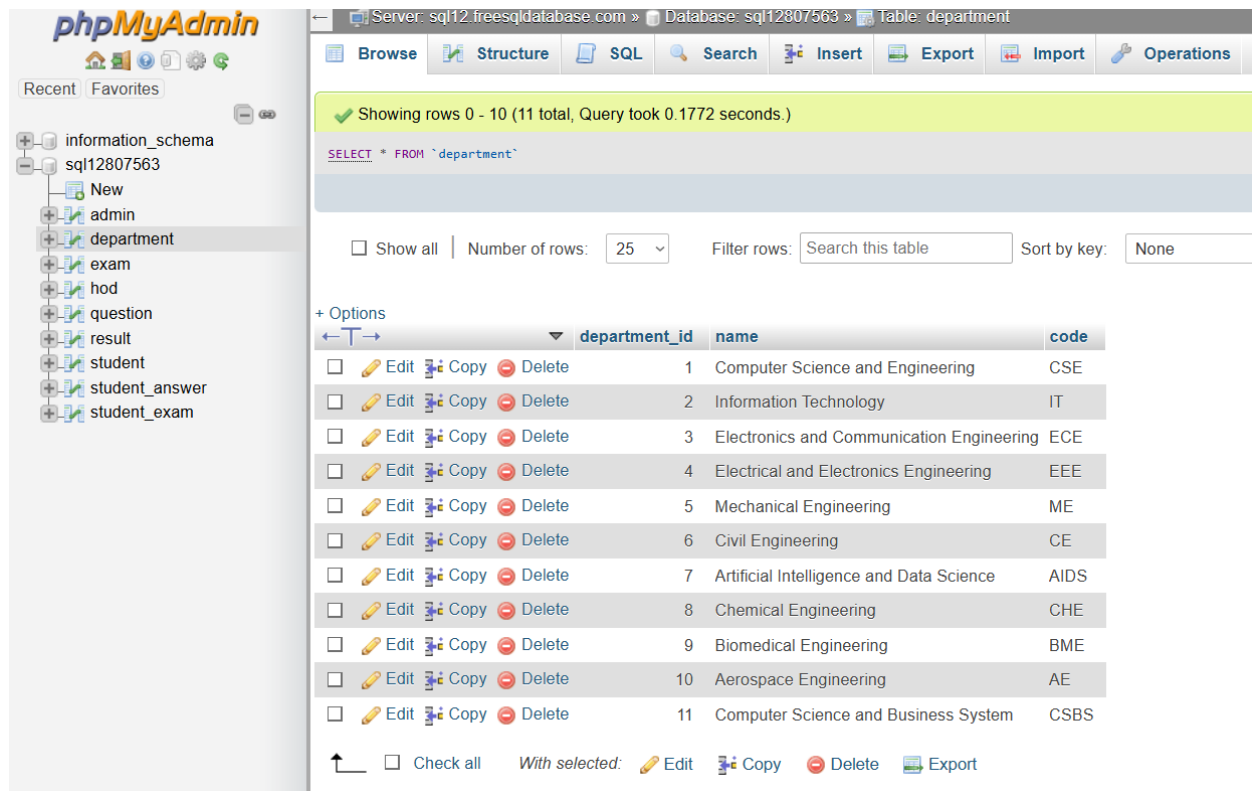
Database

- **MySQL**
- **mysql2 / Prisma / Drizzle** (whichever you used; add only the one your project uses)

Development Tools

- **Visual Studio Code**
- **Postman** for API testing
- **Git & GitHub** for version control
- **DIA Tool** for ERD and relational diagrams

Filters									
Containing the word:									
	Table	Action	Rows	Type	Collation	Size	Overhead		
☐	admin	★ Browse Structure Search Insert Empty Drop	1	InnoDB	latin1_swedish_ci	32 KiB	-		
☐	department	★ Browse Structure Search Insert Empty Drop	11	InnoDB	latin1_swedish_ci	32 KiB	-		
☐	exam	★ Browse Structure Search Insert Empty Drop	4	InnoDB	latin1_swedish_ci	32 KiB	-		
☐	hod	★ Browse Structure Search Insert Empty Drop	11	InnoDB	latin1_swedish_ci	48 KiB	-		
☐	question	★ Browse Structure Search Insert Empty Drop	15	InnoDB	latin1_swedish_ci	32 KiB	-		
☐	result	★ Browse Structure Search Insert Empty Drop	2	InnoDB	latin1_swedish_ci	64 KiB	-		
☐	student	★ Browse Structure Search Insert Empty Drop	46	InnoDB	latin1_swedish_ci	48 KiB	-		
☐	student_answer	★ Browse Structure Search Insert Empty Drop	12	InnoDB	latin1_swedish_ci	48 KiB	-		
☐	student_exam	★ Browse Structure Search Insert Empty Drop	9	InnoDB	latin1_swedish_ci	48 KiB	-		
	9 tables	Sum	111	InnoDB	latin1_swedish_ci	384 KiB	0 B		
↑	☐ Check all	With selected:							



5.3 Frontend and Backend Description

5.3.1 Frontend Description

The frontend is built using **Next.js and React**, providing a responsive and user-friendly interface.

Key UI Features:

- **Admin Dashboard**
 - Manage departments, students, exams
 - Edit/delete operations
 - View results
- **Student Dashboard**
 - View upcoming exams
 - Start exam interface with timer
 - Navigation between questions
- **Result Page**
 - Complete score analysis
 - Summary of correct and incorrect answers

5.3.2 Backend Description

The backend is implemented using **Next.js API Routes** acting as the server layer. It handles:

- Authentication (JWT generation & verification)
- CRUD operations for all entities
- Exam timer validation and submission
- Automated result calculation
- Database queries and integrity enforcement

Core Backend Modules:

- **auth module**
- **exam management module**
- **question controller**
- **student exam handler**
- **result generator**

NOV 16 12:59:47.56	POST	---	pegasus-antkin...		/admin/exams/1
NOV 16 12:59:46.71	GET	200	pegasus-antkin...		/_next/static/chunks/fb5263ced28562ee.js
NOV 16 12:59:46.20	GET	200	pegasus-antkin...		/admin/exams/1
NOV 16 12:59:45.62	GET	200	pegasus-antkin...		/admin/exams/1
NOV 16 12:59:44.72	GET	---	pegasus-antkin...		/admin/exams/1
NOV 16 12:59:44.55	POST	200	pegasus-antkin...		/admin/exams
NOV 16 12:59:43.88	POST	200	pegasus-antkin...		/admin/exams
NOV 16 12:59:43.27	POST	200	pegasus-antkin...		/admin/exams
NOV 16 12:59:41.59	POST	200	pegasus-antkin...		/admin/dashboard
NOV 16 12:59:40.86	POST	200	pegasus-antkin...		/admin/dashboard
NOV 16 12:59:40.20	POST	200	pegasus-antkin...		/admin/dashboard
NOV 16 12:59:39.52	POST	200	pegasus-antkin...		/admin/dashboard
NOV 16 12:59:38.58	POST	200	pegasus-antkin...		/admin/exams
NOV 16 12:59:37.89	POST	200	pegasus-antkin...		/admin/exams
NOV 16 12:59:36.96	POST	200	pegasus-antkin...		/admin/exams
NOV 16 12:59:35.91	POST	200	pegasus-antkin...		/admin/dashboard
NOV 16 12:59:35.22	POST	200	pegasus-antkin...		/admin/dashboard



Time	Status	Host	Request	Messages
NOV 16 12:59:37.89	POST 200	pegasus-antkin...	/admin/exams	
NOV 16 12:59:36.96	POST 200	pegasus-antkin...	/admin/exams	
NOV 16 12:59:35.91	POST 200	pegasus-antkin...	/admin/dashboard	
NOV 16 12:59:35.22	POST 200	pegasus-antkin...	/admin/dashboard	
NOV 16 12:59:34.52	POST 200	pegasus-antkin...	/admin/dashboard	
NOV 16 12:59:33.82	POST 200	pegasus-antkin...	/admin/dashboard	
NOV 16 12:59:31.86	GET 200	pegasus-antkin...	/admin/dashboard	
NOV 16 12:59:31.86	GET 200	pegasus-antkin...	/admin/dashboard	
NOV 16 12:59:31.10	GET 200	pegasus-antkin...	/admin/dashboard	
NOV 16 12:59:31.10	POST 200	pegasus-antkin...	/admin/results	
NOV 16 12:59:27.80	POST 200	pegasus-antkin...	/admin/dashboard	
NOV 16 12:59:27.58	GET 200	pegasus-antkin...	/_next/static/chunks/d61e5d14aaabf993.js	
NOV 16 12:59:27.57	GET 200	pegasus-antkin...	/_next/static/chunks/8e27d1b125c96e47.js	
NOV 16 12:59:27.55	GET 200	pegasus-antkin...	/_next/static/chunks/34fe403b34a2fd8f.js	
NOV 16 12:59:27.14	POST 200	pegasus-antkin...	/admin/dashboard	
NOV 16 12:59:26.95	GET 200	pegasus-antkin...	/admin/exams	

Logs

POST /admin/dashboard 200 ^ v X

Request started NOV 16
12:59:26.53 GMT+5:30

Request ID 8xwxb-1763278166534-0d9b21985...

Path /admin/dashboard

Host pegasus-antkin.vercel.app

User Agent Mozilla/5.0 (Windows NT 10.0;...

Received in Mumbai, India (bom1)

Firewall Allowed

Middleware 200

Execution Duration / Maximum 7ms / 5m

External APIs No outgoing requests

Fluid 106 MB

Edge Requests

2XX 3XX 4XX



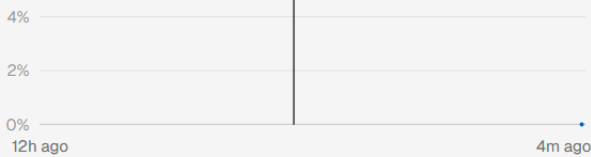
Fast Data Transfer

Outgoing Incoming
10MB 213kB



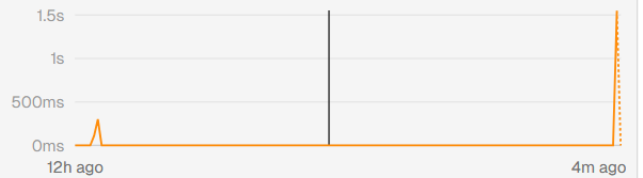
Vercel Functions

Error Timeout
0% 0%



Compute

Active CPU
1.96s



5.4 Code Snippets

Login Code

```
if (!loginPassword.length || !loginEmail) return;

setLoginLoader(true)

const res = await fetch(`${process.env.NEXT_PUBLIC_URL!}/api/auth/login`, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    email: loginEmail,
    password: loginPassword,
    role: userType
  })
})
const loginData = await res.json()
if (loginData.success === true) {
  toast.success("Success")
  if (loginData.decodedAccessToken.role === "admin") {
    router.push("/admin/dashboard")
  }
  if (loginData.decodedAccessToken.role === "student") {
    router.push("/student")
  }
  return
}
else if (loginData.error === "Invalid password") {
  toast.error("Wrong Password")
}
else if (loginData.error === "User not found") {
  toast.error(`${userType === "admin" ? "Admin" : "Student"} not found`)
}
setLoginLoader(false)
```

JWT Login Code

```
export async function POST(req: NextRequest) {
  try {
    const { email, password, role } = await req.json();
```

```

console.log(email, password, role)

const table = role === "admin" ? "admin" : "student";
const [rows]: any = await db.execute(`SELECT * FROM ${table} WHERE email = ?`,
[email]);
if (rows.length === 0) return NextResponse.json({ error: "User not found" }, { status: 404 });

const user = rows[0];
console.log(user)
const match = await bcrypt.compare(password, user.password_hash);
if (!match) return NextResponse.json({ error: "Invalid password" }, { status: 401 });

const accessToken = jwt.sign(
  { id: user.admin_id || user.student_id, role, email },
  JWT_SECRET,
  { expiresIn: "15m" }
);

const refreshToken = jwt.sign(
  { id: user.admin_id || user.student_id, role, email },
  REFRESH_SECRET,
  { expiresIn: "7d" }
);

const decodedAccessToken = jwt.verify(accessToken, process.env.JWT_SECRET!)
const res = NextResponse.json({ success: true, decodedAccessToken });

res.cookies.set("accessToken", accessToken, {
  httpOnly: true,
  secure: true,
  sameSite: "strict",
  maxAge: 15 * 60 * 1000,
});

res.cookies.set("refreshToken", refreshToken, {
  httpOnly: true,
  secure: true,
  sameSite: "strict",
  maxAge: 7 * 24 * 60 * 60,
});
return res;
}
catch (err) {
  console.error("Login error:", err);
}

```

```

    return NextResponse.json({ error: "Internal Server Error" }, { status: 500 });
  }
}

```

Department Actions Code

```

export async function getDepartments() {
  try {
    const [rows] = await db.query('SELECT * FROM department ORDER BY name ASC');
    return rows ? rows : [];
  } catch (error) {
    console.error('Error fetching departments:', error);
    throw new Error('Failed to fetch departments');
  }
}

```

```

export async function createDepartment(name: string, code: string) {
  if (!name || !code) throw new Error('Name and code are required');

  try {
    const [result]: any = await db.query(
      'INSERT INTO department (name, code) VALUES (?, ?)',
      [name, code]
    );

    return {
      department_id: result.department_id,
      name,
      code,
    };
  } catch (error: any) {
    if (error.code === 'ER_DUP_ENTRY') {
      return ({ message: "code_already_exists" })
    }
    console.error('Error creating department:', error);
    throw new Error('Failed to create department');
  }
}

```

```

export async function getDepartmentTableData() {
  try {
    const [rows] = await db.query(
      SELECT

```

```

        d.department_id,
        d.name AS name,
        d.code,
        h.name AS hod_name,
        h.email AS hod_email,
        COUNT(s.student_id) AS student_count
    FROM department d
    LEFT JOIN hod h ON h.department_id = d.department_id
    LEFT JOIN student s ON s.department_id = d.department_id
    GROUP BY d.department_id, d.name, d.code, h.name
    ORDER BY d.name ASC;
`);

    return Array.isArray(rows) ? rows : [];
} catch (error) {
    console.error('✖ Error fetching department table data:', error);
    throw new Error('Failed to fetch department table data');
}
}

```

Exam Actions Code

```

export async function getExamCount(): Promise<number> {
    try {
        const [rows]: any = await db.query("SELECT COUNT(*) AS count FROM exam");
        return rows[0].count;
    } catch (error) {
        console.error("Error fetching exam count:", error);
        throw new Error("Failed to get exam count");
    }
}

export async function getExamManagerTableData() {
    try {
        const [rows] = await db.query(`
SELECT
    e.exam_id,
    e.title AS exam_name,
    e.subject,
    d.name AS department_name,
    e.total_marks,

```

```

    e.start_time,
    e.end_time,
    COUNT(DISTINCT se.student_id) AS total_students_enrolled
FROM exam e
JOIN department d ON e.department_id = d.department_id
LEFT JOIN student_exam se ON e.exam_id = se.exam_id
GROUP BY
    e.exam_id, e.title, e.subject, d.name, e.total_marks, e.start_time, e.end_time
ORDER BY e.start_time;

```

```

`);

```

```

    return rows;
  } catch (error) {
    console.error("Error fetching all exam summaries:", error);
    throw new Error("Failed to fetch exam summaries");
  }
}

```

```

export async function createExam(formData: {
  department_id: number,
  title: string,
  description: string,
  subject: string
}) {
  try {
    const [result] = await db.execute(
      `
        INSERT INTO exam (department_id, title, description, subject, total_marks, start_time,
        end_time, duration_minutes, is_live)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
      `,
      [
        formData.department_id,
        formData.title,
        formData.description,
        formData.subject,
        0,
        new Date(),
        new Date(),
        60,
        0
      ]
    );
  }
}

```



```

    return {
      success: true,
      exam_id: (result as any).insertId,
      message: "Exam created successfully",
    };
  } catch (error: any) {
    console.error("Error creating exam:", error);
    return {
      success: false,
      message: "Failed to create exam",
      error: error.message,
    };
  }
}

export async function deleteExam(exam_id: number) {
  try {

    const [result] = await db.execute("DELETE FROM exam WHERE exam_id = ?", [exam_id])

    return {
      success: true,
      message: "Exam deleted successfully",
      affectedRows: (result as any).affectedRows,
    }
  } catch (error: any) {
    console.error("Error deleting exam:", error)
    return {
      success: false,
      message: "Failed to delete exam",
      error: error.message,
    }
  }
}

```

// ♦ *Get exam and questions*

```

export async function getExamWithQuestions(examId: number) {
  const [examRows]: any = await db.execute(
    "SELECT * FROM exam WHERE exam_id = ?",
    [examId]
  )
  const [questionRows]: any = await db.execute(
    "SELECT * FROM question WHERE exam_id = ?",

```

```

    [examId]
  )

  return {
    exam: examRows[0],
    questions: questionRows,
  }
}

// ♦ Update exam details
export async function updateExamDetails(
  examId: number,
  data: {
    title: string
    description: string
    total_marks: number
    start_time: string
    end_time: string
    duration_minutes: number
  }
) {
  await db.execute(
    `UPDATE exam
    SET title=?, description=?, total_marks=?, start_time=?, end_time=?, duration_minutes=?
    WHERE exam_id=?`,
    [
      data.title,
      data.description,
      data.total_marks,
      data.start_time,
      data.end_time,
      data.duration_minutes,
      examId,
    ]
  )

  return { success: true, message: "Exam updated successfully" }
}

export async function addQuestion(question: questionType) {
  try {
    const query = `
      INSERT INTO question (
        exam_id, question_text, option_a, option_b, option_c, option_d, correct_option
      ) VALUES (?, ?, ?, ?, ?, ?, ?)
    `;
  }
}

```

```

const values = [
  question.exam_id,
  question.question_text,
  question.option_a,
  question.option_b,
  question.option_c,
  question.option_d,
  question.correct_option,
];

const [result]: any = await db.execute(query, values);

return {
  success: true,
  insertedId: result.insertId, //  MySQL auto-generated ID
  message: "Question added successfully",
};
} catch (error: any) {
  console.error("Error adding question:", error);
  return { success: false, message: error.message };
}
}

export async function saveQuestions(questions: questionType[]) {
  try {
    if (!questions || questions.length === 0) {
      return { success: false, message: "No questions to save." }
    }

    // separate new and existing questions
    const newQuestions = questions.filter((q) => !q.question_id)
    const existingQuestions = questions.filter((q) => q.question_id)

    //  Insert new questions
    const insertedIds: number[] = []
    if (newQuestions.length > 0) {
      for (const q of newQuestions) {
        const [res] = await db.execute(
          `
          INSERT INTO question
          (exam_id, question_text, option_a, option_b, option_c, option_d, correct_option)
          VALUES (?, ?, ?, ?, ?, ?, ?)
          `,
          [
            q.exam_id,

```

```

        q.question_text,
        q.option_a,
        q.option_b,
        q.option_c,
        q.option_d,
        q.correct_option,
    ]
)
insertedIds.push((res as any).insertId)
}
}

//  Update existing questions
if (existingQuestions.length > 0) {
    for (const q of existingQuestions) {
        await db.execute(
            `
            UPDATE question
            SET question_text = ?, option_a = ?, option_b = ?, option_c = ?, option_d = ?, correct_option
= ?
            WHERE question_id = ? AND exam_id = ?
            `,
            [
                q.question_text,
                q.option_a,
                q.option_b,
                q.option_c,
                q.option_d,
                q.correct_option,
                q.question_id,
                q.exam_id,
            ]
        )
    }
}

return {
    success: true,
    message: `Saved ${questions.length} questions successfully.`,
    insertedIds,
}
} catch (error: any) {
    console.error("Error saving questions:", error)
    return {
        success: false,
        message: "Failed to save questions.",
    }
}

```

```

        error: error.message,
    }
}
}

```

// ♦ *Delete question*

```

export async function deleteQuestion(questionId: number) {
    await db.execute("DELETE FROM question WHERE question_id = ?", [questionId])
    return { success: true, message: "Question deleted" }
}

```

```

export async function saveExam(exam: examType) {

```

```

    const conn = db;

```

```

    try {

```

```

        // ✅ Create new exam

```

```

        if (!exam.exam_id) {

```

```

            const [res]: any = await conn.execute(

```

```

                `INSERT INTO exam (department_id, title, description, total_marks, start_time, end_time,
duration_minutes)

```

```

                VALUES (?, ?, ?, ?, ?, ?, ?)`,

```

```

                [

```

```

                    exam.department_id,

```

```

                    exam.title,

```

```

                    exam.description,

```

```

                    exam.total_marks,

```

```

                    exam.start_time,

```

```

                    exam.end_time,

```

```

                    exam.duration_minutes,

```

```

                ]

```

```

            );

```

```

            exam.exam_id = res.insertId;

```

```

        }

```

```

    else {

```

```

        await conn.execute(

```

```

            `UPDATE exam

```

```

                SET department_id=?, title=?, description=?, total_marks=?, start_time=?, end_time=?,
duration_minutes=?

```

```

                WHERE exam_id=?`,

```

```

                [

```

```

                    exam.department_id,

```

```

                    exam.title,

```

```

                    exam.description,

```

```

                    exam.total_marks,

```

```

                    exam.start_time,

```

```

                    exam.end_time,

```

```

        exam.duration_minutes,
        exam.exam_id,
    ]
    );
}

return { success: true, message: "Exam saved successfully", exam_id: exam.exam_id };
} catch (error: any) {
    console.error("✖ Error saving exam:", error);
    return { success: false, message: "Error saving exam" };
}
}

export async function getActiveExams() {
    const [rows] = await db.query(
        `
        SELECT * FROM exam
        WHERE start_time <= NOW()
        AND end_time >= NOW();
        `
    );

    return rows;
}

export async function initiateExam(studentId: number, examId: number) {
    const startTime = new Date();
    const [exam] = await db.query(
        `SELECT duration_minutes FROM exam WHERE exam_id = ?`,
        [examId]
    );

    if (exam.length === 0) throw new Error('Exam not found');

    const endTime = new Date(startTime.getTime() + exam[0].duration_minutes * 60000);

    const [result] = await db.query(
        `INSERT INTO student_exam (student_id, exam_id, start_time, end_time, status)
        VALUES (?, ?, ?, ?, 'in-progress')`,
        [studentId, examId, startTime, endTime]
    );

    return { student_exam_id: result.insertId, start_time: startTime, end_time: endTime };
}

```

```

// -----
// 8 GET LIVE EXAMS (for students to see ongoing/upcoming)
// -----
export async function getAvailableExams(departmentId: number) {
  const now = new Date();
  const [exams] = await db.query(
    `SELECT * FROM exam
     WHERE department_id = ? AND start_time <= ? AND end_time >= ?`,
    [departmentId, now, now]
  );
  return exams;
}

// -----
// 9 HOD OR ADMIN OVERVIEW (see all results of a department)
// -----
export async function getDepartmentResults(departmentId: number) {
  const [rows] = await db.query(
    `SELECT s.name AS student_name, e.title AS exam_title,
     r.correct_answers, r.total_questions, r.total_marks, r.percentage
     FROM result r
     JOIN student_exam se ON r.student_exam_id = se.student_exam_id
     JOIN student s ON se.student_id = s.student_id
     JOIN exam e ON se.exam_id = e.exam_id
     WHERE e.department_id = ?
     ORDER BY e.start_time DESC`,
    [departmentId]
  );
  return rows;
}

// -----
// 10 CLEANUP OLD EXAMS (optional maintenance task)
// -----
export async function cleanupExpiredExams() {
  await db.query(
    `DELETE FROM exam WHERE end_time < DATE_SUB(NOW(), INTERVAL 1 YEAR)`
  );
}

export async function publishExam(exam_id: number) {
  try {
    // Step 1: Get the department of this exam
    const [examRows]: any = await db.query(
      "SELECT department_id FROM exam WHERE exam_id = ?",
      [exam_id]
    );
  }
}

```

```

);

if (examRows.length === 0) {
  return { success: false, message: "Exam not found" };
}

const department_id = examRows[0].department_id;

// Step 2: Get all students in that department
const [students]: any = await db.query(
  "SELECT student_id FROM student WHERE department_id = ?",
  [department_id]
);

if (students.length === 0) {
  return { success: false, message: "No students found for this department" };
}

// Step 3: Insert student_exam records for all students
const now = new Date();
const studentExamData = students.map((s: any) => [
  s.student_id,
  exam_id,
  now,
  null,
  0,
  "not_started",
]);

const [result]: any = await db.query(
  `
  INSERT INTO student_exam (student_id, exam_id, start_time, end_time, marks_obtained,
status)
  VALUES ?
  ON DUPLICATE KEY UPDATE exam_id = exam_id
  `,
  [studentExamData]
);

await db.query(
  "UPDATE exam SET is_live = TRUE WHERE exam_id = ?",
  [exam_id]
);

return {
  success: true,

```



```

    inserted: result.affectedRows,
    message: `Exam published and assigned to ${students.length} students`,
  };
} catch (error) {
  console.error("Error publishing exam:", error);
  return { success: false, message: "Failed to publish exam" };
}
}

```

```

export async function submitExam(student_id: number, exam_id: number) {
  try {
    await db.query(
      `UPDATE student_exam
      SET status = 'completed', end_time = NOW()
      WHERE student_id = ? AND exam_id = ?`,
      [student_id, exam_id]
    );

    await generateResult(student_id, exam_id);

    return { success: true, message: "Exam submitted successfully" };
  } catch (error) {
    console.error("Error submitting exam:", error);
    return { success: false, message: "Failed to submit exam" };
  }
}

```

```

export async function generateResult(student_id: number, exam_id: number) {
  try {
    // Fetch total marks for this exam
    const [[exam]]: any = await db.query(
      "SELECT total_marks FROM exam WHERE exam_id = ?",
      [exam_id]
    );

    // Count total and correct answers
    const [[answers]]: any = await db.query(
      `SELECT COUNT(*) AS total_questions,
      SUM(is_correct) AS correct_answers
      FROM student_answer
      WHERE student_id = ?
      AND question_id IN (SELECT question_id FROM question WHERE exam_id = ?)`,
      [student_id, exam_id]
    );

    const total_questions = answers.total_questions || 0;

```

```

const correct_answers = answers.correct_answers || 0;
const marks_per_question =
  total_questions > 0 ? exam.total_marks / total_questions : 0;
const total_obtained = correct_answers * marks_per_question;
const percentage =
  exam.total_marks > 0
    ? (total_obtained / exam.total_marks) * 100
    : 0;

```

```

// Get student_exam_id
const [[studentExam]]: any = await db.query(
  `SELECT student_exam_id FROM student_exam
  WHERE student_id = ? AND exam_id = ?`,
  [student_id, exam_id]
);

```

```

if (!studentExam) {
  throw new Error("Student exam record not found.");
}

```

```

const student_exam_id = studentExam.student_exam_id;

```

```

// Insert or update result table
await db.query(
  `INSERT INTO result (
    student_id,
    exam_id,
    student_exam_id,
    total_questions,
    correct_answers,
    total_marks,
    percentage
  )
  VALUES (?, ?, ?, ?, ?, ?, ?)
  ON DUPLICATE KEY UPDATE
    total_questions = VALUES(total_questions),
    correct_answers = VALUES(correct_answers),
    total_marks = VALUES(total_marks),
    percentage = VALUES(percentage)`,
  [
    student_id,
    exam_id,
    student_exam_id,
    total_questions,
    correct_answers,
    total_obtained,
    percentage
  ]
);

```

```

        percentage,
    ]
);

return {
    success: true,
    marks: total_obtained,
    percentage,
};
} catch (error: any) {
    console.error("Error generating result:", error);
    return { success: false, message: error.message };
}
}

export async function saveAnswer(student_id: number, question_id: string, selected_option:
string) {
    try {
        const [[question]]: any = await db.query(
            "SELECT correct_option FROM question WHERE question_id = ?",
            [question_id]
        );

        const is_correct = question.correct_option === selected_option ? 1 : 0;

        await db.query(
            `INSERT INTO student_answer (student_id, question_id, selected_option, is_correct)
            VALUES (?, ?, ?, ?)
            ON DUPLICATE KEY UPDATE selected_option = VALUES(selected_option), is_correct =
VALUES(is_correct)`,
            [student_id, question_id, selected_option, is_correct]
        );

        return { success: true, correct: is_correct };
    } catch (error) {
        console.error("Error saving answer:", error);
        return { success: false, message: "Failed to save answer" };
    }
}

export async function startExam(student_id: number, exam_id: number) {
    try {
        const [result]: any = await db.query(
            `UPDATE student_exam
            SET status = 'in_progress', start_time = NOW()
            WHERE student_id = ? AND exam_id = ?`,

```

```

    [student_id, exam_id]
  );
  return { success: true, message: "Exam started" };
} catch (error) {
  console.error(error);
  return { success: false, message: "Failed to start exam" };
}
}

```

```

export async function getStudentExams(studentId: number) {
  try {
    const query = `
      SELECT
        se.student_exam_id,
        e.exam_id,
        e.title AS exam_title,
        e.description AS exam_description,
        e.subject,
        e.total_marks,
        e.duration_minutes,
        e.start_time,
        e.end_time,
        se.status,
        se.marks_obtained,
        e.title
      FROM student_exam se
      JOIN exam e ON se.exam_id = e.exam_id
      WHERE se.student_id = ?
      ORDER BY e.start_time DESC
    `;

    const [rows] = await db.execute(query, [studentId]);
    return {
      success: true,
      exams: rows,
    };
  } catch (error: any) {
    console.error("Error fetching student exams:", error);
    return { success: false, message: error.message };
  }
}

```

```

export async function getUpcomingExams(student_id: number) {

```

```

try {
  const query = `
    SELECT
      e.exam_id,
      e.title,
      e.description,
      e.total_marks,
      e.start_time,
      e.end_time,
      e.duration_minutes,
      e.subject,
      d.name AS department_name
    FROM exam e
    JOIN department d ON e.department_id = d.department_id
    JOIN student s ON s.department_id = e.department_id
    WHERE s.student_id = ?
    AND e.is_live = TRUE
    AND e.start_time > NOW()
    ORDER BY e.start_time ASC
  `;

  const [rows] = await db.query(query, [student_id]);

  return {
    success: true,
    upcomingExams: rows,
  };
} catch (error: any) {
  console.error("Error fetching upcoming exams:", error);
  return { success: false, message: error.message };
}
}

```

```

export async function getExamDetails(exam_id: number) {
  try {
    const query = `
      SELECT
        e.exam_id,
        e.title,
        e.description,
        e.total_marks,
        e.duration_minutes,
        e.subject,
        e.start_time,

```

```

        e.end_time,
        d.name AS department_name
    FROM exam e
    JOIN department d ON e.department_id = d.department_id
    WHERE e.exam_id = ?
`;

const [rows]: any = await db.query(query, [exam_id]);
if (!rows.length) {
    return { success: false, message: "Exam not found" };
}

return {
    success: true,
    exam: rows[0],
};
} catch (error: any) {
    console.error("Error fetching exam details:", error);
    return { success: false, message: error.message };
}
}

export async function getExamQuestions(exam_id: number) {
    try {
        const query = `
            SELECT
                question_id,
                exam_id,
                question_text,
                option_a,
                option_b,
                option_c,
                option_d
            FROM question
            WHERE exam_id = ?
            ORDER BY question_id ASC
        `;

        const [rows]: any = await db.query(query, [exam_id]);
        return {
            success: true,
            questions: rows,
        };
    } catch (error: any) {
        console.error("Error fetching exam questions:", error);
        return { success: false, message: error.message };
    }
}

```

```
}  
}
```

```
export async function getStudentResults(student_id: number) {  
  try {  
    const [rows]: any = await db.query(  
      ,  
      SELECT  
        e.exam_id,  
        e.title,  
        e.subject,  
        e.total_marks,  
        r.total_marks AS marks_obtained,  
        r.percentage,  
        se.end_time AS submitted_at  
      FROM result r  
      JOIN exam e ON r.exam_id = e.exam_id  
      JOIN student_exam se ON r.student_exam_id = se.student_exam_id  
      WHERE se.student_id = ?  
      ORDER BY se.end_time DESC  
      ,  
      [student_id]  
    );  
  
    return {  
      success: true,  
      results: rows,  
    };  
  } catch (error: any) {  
    console.error("Error fetching student results:", error);  
    return { success: false, message: error.message };  
  }  
}
```

```
export async function getResultsGroupedByExam() {  
  try {  
    const [rows]: any = await db.query(  
      ,  
      SELECT  
        e.exam_id,  
        e.title AS exam_title,  
        e.subject,  
        e.total_marks AS max_marks,  
        s.student_id,  
        s.name AS student_name,
```

```

        r.total_marks AS marks_obtained,
        r.percentage,
        se.end_time AS submitted_at
    FROM result r
    JOIN exam e ON r.exam_id = e.exam_id
    JOIN student_exam se ON r.student_exam_id = se.student_exam_id
    JOIN student s ON se.student_id = s.student_id
    ORDER BY e.exam_id, s.name
    ,
);

// Group results by exam_id
const groupedResults: Record<number, any> = {};
for (const row of rows) {
    if (!groupedResults[row.exam_id]) {
        groupedResults[row.exam_id] = {
            exam_id: row.exam_id,
            exam_title: row.exam_title,
            subject: row.subject,
            max_marks: row.max_marks,
            results: [],
        };
    }
    groupedResults[row.exam_id].results.push({
        student_id: row.student_id,
        student_name: row.student_name,
        marks_obtained: row.marks_obtained,
        percentage: row.percentage,
        submitted_at: row.submitted_at,
    });
}

// Convert grouped object to array for easy rendering
const finalResults = Object.values(groupedResults);

return {
    success: true,
    groupedResults: finalResults,
};
} catch (error: any) {
    console.error("Error fetching grouped results:", error);
    return { success: false, message: error.message };
}
}

```


Student Actions Code

```
export async function getStudentCount(departmentId?: number) {
  try {
    const query = departmentId
      ? "SELECT COUNT(*) AS count FROM student WHERE department_id = ?"
      : "SELECT COUNT(*) AS count FROM student";
    const [rows]: any = await db.query(query, departmentId ? [departmentId] : []);
    return rows[0].count as number;
  } catch (error) {
    console.error("Error counting students:", error);
    return 0;
  }
}

export async function insertStudents(students: studentType[]) {
  try {
    if (!students || students.length === 0) {
      throw new Error("No students provided");
    }

    // Hash passwords in parallel
    const studentData = await Promise.all(
      students.map(async (student) => {
        const hashedPassword = await bcrypt.hash(student.student_id.toString(), 10);
        return [
          student.student_id,
          student.name,
          student.email,
          student.department_id,
          hashedPassword,
        ];
      })
    );

    // Construct SQL dynamically to avoid `VALUES ?` issues with ON DUPLICATE
    const placeholders = studentData
      .map(() => "(?, ?, ?, ?, ?)")
      .join(", ");
    const flatValues = studentData.flat();

    const sql = `
```

```

INSERT INTO student (student_id, name, email, department_id, password_hash)
VALUES ${placeholders}
ON DUPLICATE KEY UPDATE
  name = VALUES(name),
  email = VALUES(email),
  department_id = VALUES(department_id),
  password_hash = VALUES(password_hash)
`;

const [result]: any = await db.query(sql, flatValues);

console.log("✅ Insert complete:", result);

return {
  success: true,
  affectedRows: result.affectedRows,
  changedRows: result.changedRows,
};
} catch (error) {
  console.error("❌ Error inserting students:", error);
  throw new Error("Failed to insert students");
}
}

export async function getStudentDepartment(studentId: number) {
  try {
    const [rows] = await db.execute(
      `
      SELECT
        d.department_id,
        d.name AS name,
        d.code AS code
      FROM student s
      INNER JOIN department d ON s.department_id = d.department_id
      WHERE s.student_id = ?;
      `,
      [studentId]
    );

    // rows will be an array of RowDataPackets
    const departments = rows as {
      department_id: number;
      name: string;
      code: string;
    }[];
  }
}

```

```

    if (departments.length === 0) {
      return { error: "Student or department not found" };
    }

    return departments[0];
  } catch (error) {
    console.error("Error fetching student department:", error);
    return { error: "Database error" };
  }
}

export async function getDepartmentStudents(deptId: number) {
  const [rows] = await db.query(
    `SELECT student_id, name, email
    FROM student
    WHERE department_id = ?`,
    [deptId]
  );

  return rows;
};

export async function saveStudentAnswer(
  student_id: number,
  question_id: number,
  selected_option: string
) {
  try {
    // Get the correct answer for this question
    const [correctRows]: any = await db.query(
      `SELECT correct_option FROM question WHERE question_id = ?`,
      [question_id]
    );

    if (!correctRows.length) {
      throw new Error("Question not found");
    }

    const correct_option = correctRows[0].correct_option;
    const is_correct = correct_option === selected_option;

    // Insert or update student answer
    const [result]: any = await db.query(
      `
      INSERT INTO student_answer (student_id, question_id, selected_option, is_correct)
      VALUES (?, ?, ?, ?)
    `
    );
  }
}

```

```

ON DUPLICATE KEY UPDATE
  selected_option = VALUES(selected_option),
  is_correct = VALUES(is_correct)
,
  [student_id, question_id, selected_option, is_correct]
);

// Get the answer_id of the inserted/updated record
const [rows]: any = await db.query(
  `SELECT answer_id FROM student_answer WHERE student_id = ? AND question_id =
?`,
  [student_id, question_id]
);

return {
  success: true,
  answer_id: rows[0].answer_id,
};
} catch (error: any) {
  console.error("Error saving student answer:", error);
  return { success: false, message: error.message };
}
}

export async function getStudentAnswers(studentId: number, examId: number) {
  try {
    const query = `
SELECT
  sa.answer_id,
  sa.question_id,
  sa.selected_option
FROM student_answer sa
JOIN question q ON sa.question_id = q.question_id
WHERE sa.student_id = ? AND q.exam_id = ?
`;

    const [rows]: any = await db.query(query, [studentId, examId]);

    return {
      success: true,
      answers: rows,
    };
  } catch (error: any) {
    console.error("Error fetching student answers:", error);
    return { success: false, message: error.message };
  }
}

```

```
}
```

User Actions Code

```
export async function getCurrentUser() {
  const accessToken = (await cookies()).get('accessToken')?.value;
  if (!accessToken) return null;

  let token = null;
  try {
    token = jwt.verify(accessToken, process.env.JWT_SECRET!)
  }
  catch (e) {
    return null;
  }

  const { id, role } = token;

  try {
    if (role === 'admin') {
      const [rows] = await db.query(
        'SELECT admin_id AS id, name, email, "admin" AS role FROM admin WHERE'
admin_id = ?',
        [id]
      );
      return Array.isArray(rows) && rows.length > 0 ? rows[0] : null;
    } else if (role === 'student') {
      const [rows] = await db.query(
        'SELECT student_id AS id, name, email, department_id, "student" AS role FROM'
student WHERE student_id = ?',
        [id]
      );
      return Array.isArray(rows) && rows.length > 0 ? JSON.parse(JSON.stringify(rows[0])) :
null;
    }
    return null;
  } catch (err) {
    console.error('Error fetching user:', err);
    return null;
  }
}

export async function getAdminCount(): Promise<number> {
  try {
```

```

    const [rows]: any = await db.query("SELECT COUNT(*) AS count FROM admin");
    return rows[0].count;
  } catch (error) {
    console.error("Error fetching admin count:", error);
    throw new Error("Failed to get admin count");
  }
}

export async function createAdmin() {
  try {
    const name = "Abhraneel Dhar";
    const email = "abhraneeldhar@gmail.com";
    const plainPassword = "helloworld";

    // Hash password securely
    const passwordHash = await bcrypt.hash(plainPassword, 10);

    const [result] = await db.execute(
      `INSERT INTO admin (name, email, password_hash) VALUES (?, ?, ?)`,
      [name, email, passwordHash]
    );

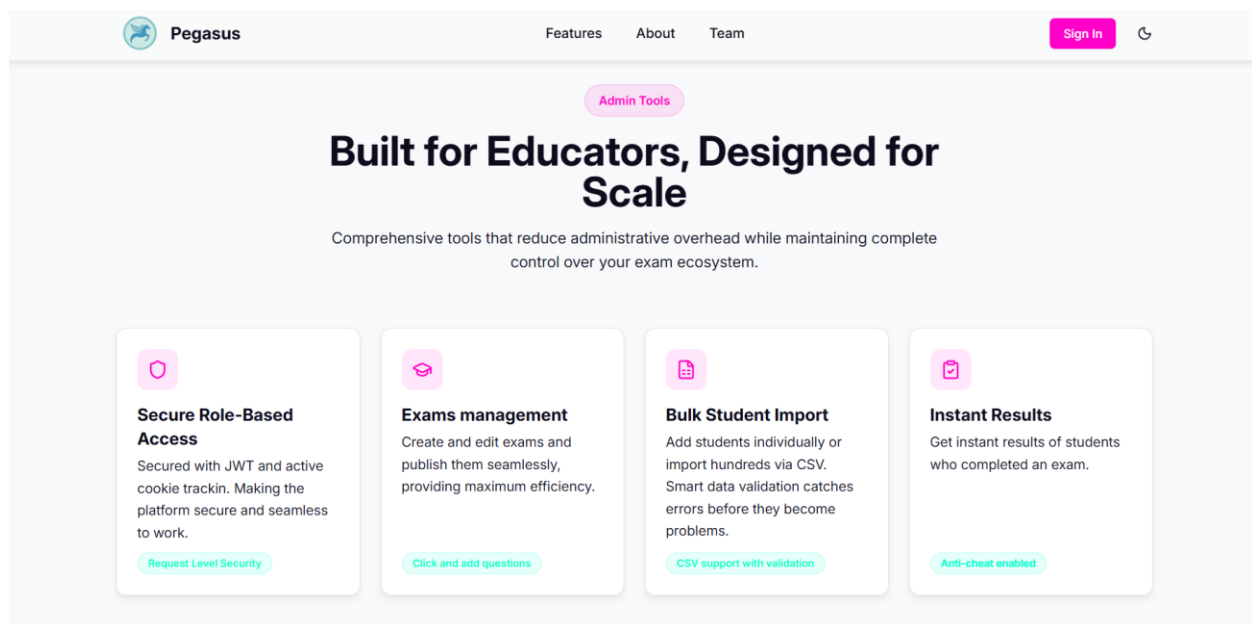
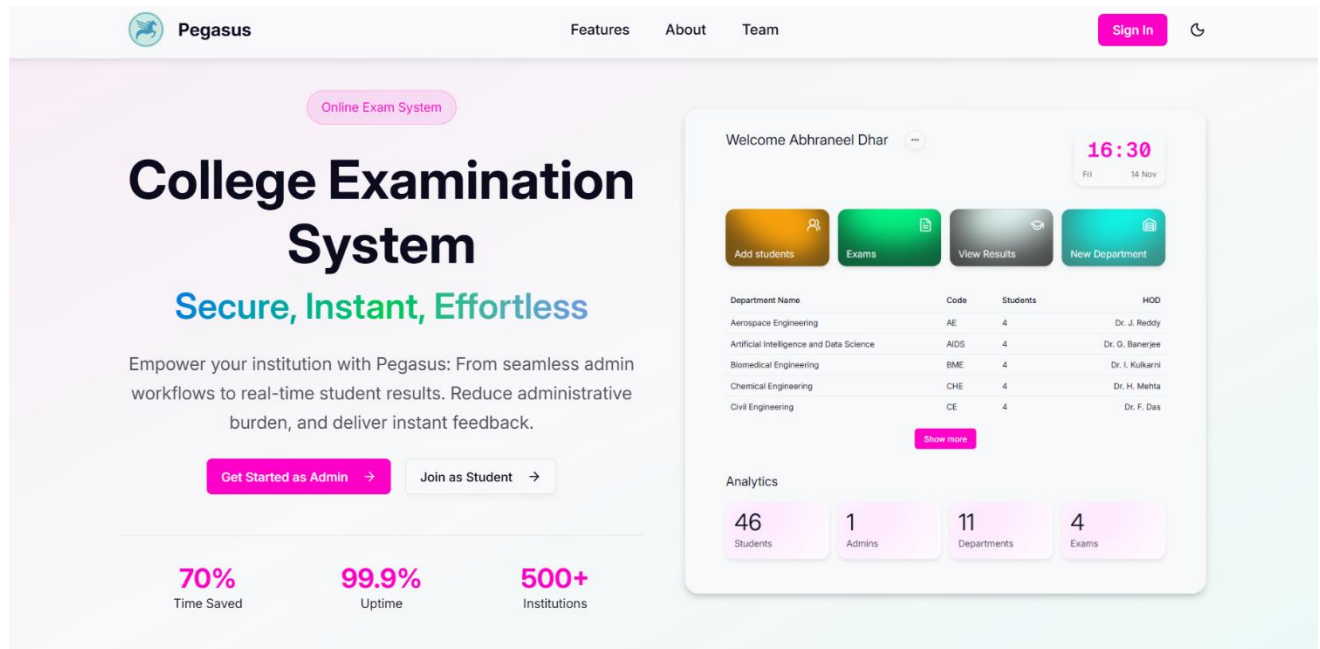
    console.log("✅ Admin created successfully:", result);
  } catch (err) {
    console.error("❌ Error creating admin:", err);
  }
}

```

6. Testing and Evaluation

6.1 Screenshots of the Application

To demonstrate the working of the College Examination System, various parts of the application were executed and tested. Screenshots of important modules are included below for verification and documentation.



Our Team

Our team collaborated effectively to design, develop, and document the College Examination System.



Abhraneel Dhar

CSBS ' 27

[Visit Profile](#)



Abhinandan Rakshit

CSBS ' 27

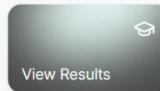
[Visit Profile](#)

Welcome Abhraneel Dhar



13:10

Sun 16 Nov



Department Name	Code	Students	HOD
Aerospace Engineering	AE	4	Dr. J. Reddy
Artificial Intelligence and Data Science	AIDS	4	Dr. G. Banerjee
Biomedical Engineering	BME	4	Dr. I. Kulkarni
Chemical Engineering	CHE	4	Dr. H. Mehta
Civil Engineering	CE	4	Dr. F. Das

[Show more](#)

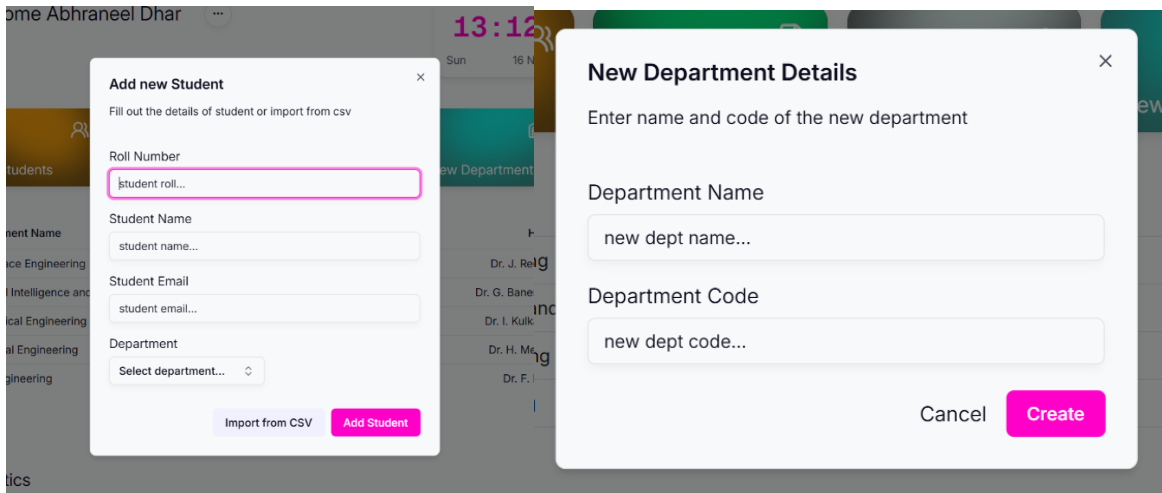
Analytics

46

1

11

4



Correct Option

C

+ Add Question

Saving an Exam

Once you've filled in all the exam details, click the **Save** button to store your exam. This ensures all your entered information is safely saved to the system.

Adding Questions

To create new questions for the exam, simply click the **Add Question** button. You can add as many questions as needed before publishing.

Deleting an Exam

If you wish to remove an existing exam, use the **Delete Exam** button. This action will permanently remove the exam from the system.

Publishing an Exam

When your exam is ready to go live, click the **Publish** button. The exam will then be visible to students and marked as Live.

Delete this exam

Question 3



Which of the following is not a programming language?

Python

Java

HTML

C++

Correct Option

C

Question 4



In C programming, what is the size of an int on most 64-bit systems?

2 bytes

4 bytes

16 bytes

8 bytes

Correct Option

Exam Details

13:11

Sun 16 Nov

Exam name
CA4 exam

Description
this is an online exam

Subject
Artificial Intelligence

Start Time
11/13/2025 00:00:00

Department
Computer Science and Busi...

Duration Total Marks
0 10

Questions

Question 1



Exam Manager

New Exam

Results

Create new exam

Enter the details of the exam and proceed

Exam name
exam name...

Description
exam description...

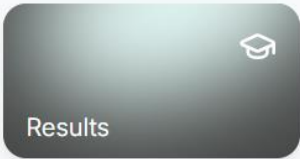
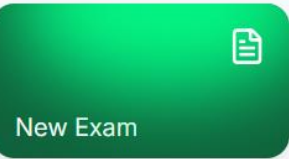
Subject
exam description...

Department
Select department...

Cancel

Create

Exam Manager



Exams Listed

Name	Dept	Subject	Total Marks	Enrolled	Action
CA4 exam	Computer Science and Business ...	Artificial Intelligence	10	2	...
test	Electrical and Electronics Engine...	sets	0	4	...
CA4 DBMS	Computer Science and Business ...	DBMS	2	3	...
CA4 exam	Computer Science and Business ...	Machine Learning	3	0	...

Results

CA4 exam

Artificial Intelligence

Total Marks: 10

Submitted: 1

Roll Number	Student Name	Marks	Percentage	Submitted At
13031123002	Abhraneel Dhar	6	60.00	13 Nov 09:27

CA4 DBMS

DBMS

Total Marks: 2

Submitted: 1

Roll Number	Student Name	Marks	Percentage	Submitted At
13031123002	Abhraneel Dhar	2	100.00	13 Nov 20:24

Civil Engineering	CE	4	Dr. F. Das
Computer Science and Business System	CSBS	3	Dr. K. Verma
Computer Science and Engineering	CSE	5	Dr. A. Sharma
Electrical and Electronics Engineering	EEE	4	Dr. D. Rao
Electronics and Communication Engineering	ECE	5	Dr. C. Nair
Information Technology	IT	5	Dr. B. Singh
Mechanical Engineering	ME	4	Dr. E. Patel

Show less

Analytics

46

Students

1

Admins

11

Departments

4

Exams

Welcome Abhraneel Dhar

...

13:10

Sun

16 Nov



Add students



Exams



View Results



New Department

Department Name	Code	Students	HOD
Aerospace Engineering	AE	4	Dr. J. Reddy
Artificial Intelligence and Data Science	AIDS	4	Dr. G. Banerjee
Biomedical Engineering	BME	4	Dr. I. Kulkarni
Chemical Engineering	CHE	4	Dr. H. Mehta
Civil Engineering	CE	4	Dr. F. Das

Show more

Analytics

46

1

11

4

6.2 Output Demonstration

This section highlights the actual outputs generated by the system during testing.

6.2.1 Admin Module Output

- Admin successfully logs in using JWT authentication.
- Admin is able to add new departments and students.
- Exam creation, editing, and deletion operations are functioning correctly.
- Questions can be added and linked to their respective exams.
- Admin can view the list of student exam attempts and final results.

6.2.2 Student Module Output

- Students can log in securely.
- Only exams from the student's department are displayed.
- The exam interface shows each question with four options.
- The timer accurately counts down and prevents late submission.
- On submission, answers are stored, evaluated, and results are generated automatically.
- Students can view their final score, total questions, correct answers, and percentage.

6.2.3 Backend Output

- API endpoints respond with correct JSON outputs (success messages, exam data, validation errors).
- JWT tokens are validated correctly for every protected route.
- Database operations execute without errors, maintaining referential integrity.

6.3 Analysis of Results

The system was thoroughly tested using both functional and non-functional test scenarios. The overall performance and correctness were evaluated as follows:

6.3.1 Functional Performance

- All modules Admin, Student, Exam, Question, Result performed their roles as intended.
- CRUD operations executed smoothly without data inconsistency.
- Automated evaluation accurately calculated scores based on correct answers.
- Timer and submission mechanisms behaved predictably under normal and edge cases.

6.3.2 Usability Evaluation

- The user interface was found to be intuitive and easy to navigate.
- Students were able to complete exams without confusion.

- Admin workflows (exam creation, management) were logical and efficient.

6.3.3 Security Evaluation

- JWT authentication ensured that unauthorized users could not access protected resources.
- Passwords were stored as secure hashes, reducing risk of credential leaks.
- Role-based access control successfully prevented cross-role interference.

6.3.4 Performance Stability

- The application handled multiple exam submissions without crashes.
- API response times remained within acceptable limits.
- Database operations were consistent and showed no deadlocks or failures.

6.3.5 Overall Evaluation

The testing confirms that the system meets all major functional requirements and provides a robust, secure, and user-friendly platform for conducting college-level examinations. The application is ready for deployment and real-world usage, with potential for further enhancements in scalability and analytics.

7. Conclusion

7.1 Summary of Achievements

The College Examination Management System successfully fulfills the primary goal of digitizing and streamlining examination-related activities within a college environment. The project implements a fully functional platform using **Next.js, TypeScript, MySQL, and JWT authentication**, ensuring both security and efficiency.

Key achievements include:

- A complete **role-based system** supporting Admin and Student functionalities.
- **Secure authentication** and authorization using JWT.
- Fully implemented **department, student, exam, and question management** modules for administrators.
- A user-friendly **exam interface** for students with accurate timer control and question navigation.
- **Automated evaluation** of answers, efficient result computation, and clean result presentation.
- A well-structured **relational database** with enforced integrity constraints and normalized schema.
- A functional full-stack architecture with smooth communication between frontend and backend.
- A scalable, maintainable, and extendable solution suitable for academic institutions.

Overall, the project demonstrates strong application of database concepts, full-stack development, and software engineering practices.

7.2 Limitations

While the system performs well for its intended scope, certain limitations remain:

- **No proctoring mechanisms** such as webcam monitoring or cheating prevention.
- **No descriptive/subjective question support** only multiple-choice questions are evaluated automatically.
- **No real-time analytics or dashboards** for performance trends or department-level insights.
- **Limited notification system** does not include email/SMS alerts for exam schedules or results.
- **Basic UI/UX** suitable for internal use but can be enhanced for large-scale deployment.
- **No offline exam support** requires active internet connection.
- **Scalability constraints** depending on hosting environment, since load balancing and caching are not yet implemented.

8. Future Scope

The current system provides essential features for conducting and managing online examinations. However, several enhancements can further improve functionality, scalability, and real-world usability:

8.1 Future Enhancements

1. **AI-Based Proctoring System**
 - Integration of webcam monitoring, face detection, and behavior analysis to prevent cheating.
2. **Support for Subjective/Descriptive Questions**
 - Allow students to upload written answers.
 - Implement manual faculty evaluation or AI-assisted scoring.
3. **Advanced Analytics Dashboard**
 - Department-wise performance metrics
 - Student progress tracking
 - Exam difficulty analysis
4. **Notification & Communication System**
 - Email/SMS alerts for exam schedules, reminders, and results.
5. **Role Expansion**
 - Add faculty roles for question creation and result verification.
 - Add super-admin for global system control.
6. **Offline Exam Mode**
 - Exams could be attempted without internet; data syncs when connection resumes.
7. **Cloud Deployment & Scalability**
 - Load balancing, caching, and distributed database support for large campuses.
8. **Mobile Application**
 - A dedicated Android/iOS app for seamless student experience.
9. **Improved UI/UX Design**
 - Professional dashboards, animations, accessibility features, dark mode.
10. **Blockchain-Based Result Verification**
 - Immutable storage of marksheets and results to prevent tampering.

9. References

- [1] J. Ullman and J. Widom, "A First Course in Database Systems," 3rd ed., Pearson, 2007.
- [2] R. Elmasri and S. Navathe, "Fundamentals of Database Systems," 7th ed., Pearson, 2016.
- [3] Next.js Documentation, "Next.js 14 - App Router and API Routes Guide." Available: <https://nextjs.org/docs>
- [4] MySQL Documentation, "MySQL 8.0 Reference Manual." Available: <https://dev.mysql.com/doc/>
- [5] JSON Web Tokens, "Introduction to JWT." Available: <https://jwt.io/introduction/>
- [6] React Documentation, "React - A JavaScript library for building user interfaces." Available: <https://react.dev/>
- [7] B. Kernighan and D. Ritchie, "The C Programming Language," 2nd ed., Prentice Hall, 1988.
- [8] MDN Web Docs, "HTTP and Web Security." Available: <https://developer.mozilla.org/>

10. Appendix

10.1 Acronyms

Acronym	Full Form
DBMS	Database Management System
ERD	Entity Relationship Diagram
JWT	JSON Web Token
CRUD	Create, Read, Update, Delete
UI	User Interface
UX	User Experience
API	Application Programming Interface
SQL	Structured Query Language
PK	Primary Key
FK	Foreign Key

10.2 Code Snippets

10.2.1 MySQL Connection Setup (Next.js)

```
import mysql from 'mysql2/promise';

export const db = await mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'password',
  database: 'examsystem'
});
```

10.2.2 JWT Token Generation

```
import jwt from 'jsonwebtoken';

export function generateToken(userId: number) {
  return jwt.sign({ id: userId }, process.env.JWT_SECRET!, {
    expiresIn: '7d',
  });
}
```

10.2.3 Admin Login API Example

```
import { db } from '@lib/db';
```

```

import bcrypt from 'bcrypt';
import { generateToken } from '@utils/jwt';

export default async function handler(req, res) {
  const { email, password } = req.body;

  const [admin] = await db.execute(
    "SELECT * FROM admin WHERE email = ?",
    [email]
  );

  if (!admin) return res.status(400).json({ message: "User not found" });

  const valid = await bcrypt.compare(password, admin.password_hash);
  if (!valid) return res.status(401).json({ message: "Invalid password" });

  const token = generateToken(admin.admin_id);
  res.json({ token });
}

```

10.2.4 Exam Result Calculation Snippet

```

const percentage = (correctAnswers / totalQuestions) * 100;

await db.execute(
  "INSERT INTO result (student_exam_id, exam_id, total_questions,
    correct_answers, total_marks, percentage) VALUES (?, ?, ?, ?, ?, ?)",
  [studentExamId, examId, totalQuestions, correctAnswers, marksObtained,
    percentage]
);

```